SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 2 – Agile Sprint Planning & User Story Workshop

Course Code:	CSA10 / CSA1023	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 2 – **Agile Sprint Planning & User Story Workshop**
Weightage:	20%	**Total Marks:**	25
CO1 Statement:	*Apply advanced methodologies and agile project management techniques to manage software projects effectively*		
Student Name:	S. Roshni	**Reg. No.:**	192525185
Date:	09-10-2026	**Duration:**	60 minutes

Code of Conduct

I S. Roshni / 192525185 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ***S.Roshni***

Annexure A – Agile Sprint Planning & User Story Workshop

Instructions to Students:

Each student is assigned an individual software project problem statement. Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

1. **Identify the Project Vision and Stakeholders**
   - Define the project's vision, objectives, and identify all key stakeholders involved.
2. **Create Epics and User Stories**
   - Break down the requirements into Epics and formulate well-structured User Stories using the format: *As a <user>, I want <feature>, so that <benefit>*.
3. **Define Acceptance Criteria**
   - Write clear and measurable acceptance criteria for each selected user story using the Given–When–Then format or equivalent.
4. **Prioritize the Product Backlog**
   - Organize and prioritize the product backlog based on business value, customer needs, and project dependencies.
5. **Plan the Sprint Backlog**
   - Select high-priority user stories for the first sprint and divide them into actionable development tasks.
6. **Estimate Effort Using Story Points**
   - Assign story points to each selected user story using an appropriate estimation technique (e.g., Fibonacci sequence or Planning Poker) and justify your estimates.
7. **Present the Sprint Goal and Expected Deliverables**
   - Define a clear sprint goal and describe the expected sprint deliverables, including the features to be completed at the end of the sprint.

Project Title: Distributed Online Collaboration Platform for Large Student Projects (Problem No. 35)

Student: S. Roshni (192525185) **Methodology:** Scrum (2-week sprints) **Estimation:** Planning Poker (Fibonacci scale)

1. Project Vision and Stakeholders

1.1 Product Vision

"For student project teams, faculty guides and project managers who struggle with duplicated work, merge conflicts, missed deadlines and scattered documentation, the **Distributed Online Collaboration Platform** is a web-based collaboration system that unifies task and milestone management, requirements and documentation, Git-based code reviews, automated testing, CI/CD, containerized development environments and live project dashboards in one place. Unlike using separate, unconnected tools, our platform gives every team consistent workflows, role-based access, full audit trails and clear progress evidence."

1.2 Project Objectives

- Manage project teams, tasks, milestones and responsibilities across many student teams.
- Support collaborative requirements and documentation with version history.
- Integrate Git version control, pull requests and code reviews.
- Provide automated testing and CI/CD pipelines for every team repository.
- Track project progress and risks through dashboards and alerts.
- Offer containerized (Docker-based) development environments for consistent setups.
- Maintain project history and audit trails as evidence for evaluation.

1.3 Success Measures

- Merge conflicts reduced by at least 30% compared with teams using ad-hoc sharing.
- At least 90% of tasks have a named owner and due date; 100% of merged code is peer-reviewed.
- New developer environment ready in under 10 minutes using containers.
- Faculty can view the status of any team in under 3 clicks.

1.4 Stakeholders

Stakeholder	Role in the Project	Key Interest / Expectation
Student Developers	Primary users; write code, documents and tests	Easy task tracking, simple Git workflow, no merge conflicts, quick environment setup
Team Leads / Scrum Masters (students)	Plan sprints, assign tasks, coordinate teams	Clear task board, workload visibility, milestone tracking
Faculty Guide / Product Owner	Defines requirements, sets priorities, approves deliverables	Project visibility, progress evidence, quality of software engineering practices

Stakeholder	Role in the Project	Key Interest / Expectation
Project Manager (Student/Faculty)	Manages multiple teams, milestones and risks	Cross-team dashboards, risk tracking, deadline alerts
Reviewers and Testers (QA)	Review pull requests, run and verify tests	Review workflow, automated test results, defect tracking
Documentation Leads	Maintain requirements, design and reports	Collaborative editing, versioning, consistent documents
DevOps / Platform Administrator	Maintains CI/CD, containers, servers and access control	Stable pipelines, secure role-based access, monitoring and backups
Institution / Department	Sponsor of the academic programme	Better project outcomes, standardised project-management evidence
External Evaluators / Industry Mentors	Assess final project and give feedback	Audit trails, project history and demonstrable deliverables

1.5 Stakeholder Influence and Interest Analysis

Stakeholder	Influence	Interest	Engagement Strategy
Faculty Guide / Product Owner	High	High	Manage closely: weekly backlog refinement and sprint review
Student Developers	Medium	High	Keep involved: daily scrum, planning poker, retrospectives
Team Leads / Scrum Masters	Medium	High	Keep involved: sprint planning and impediment removal
Project Manager	High	Medium	Keep satisfied: dashboards and risk reports
DevOps / Platform Administrator	Medium	Medium	Consult on infrastructure, security and pipeline decisions
Reviewers and Testers	Low	High	Keep informed: review queues and test summaries
Institution / Department	High	Low	Keep satisfied: end-of-sprint progress reports
External Evaluators / Mentors	Low	Low	Monitor: share demos and final audit evidence

1.6 User Personas

Persona	Description	Main Pain Points	Main Goals
Arun – Student Developer	2nd-year student in a team of 6; writes code and tests	Overwritten code, unclear tasks, slow environment set-up	Know exactly what to do, merge safely, start coding fast
Meena – Team Lead	Coordinates a team and reports to faculty	Tracking work by chat/spreadsheets, missed deadlines	One board for tasks, reviews and status
Dr. Rajesh – Faculty Guide	Guides several teams in a large project	No single view of progress or contribution	Live dashboards and trustworthy evidence for evaluation

Persona	Description	Main Pain Points	Main Goals
Kiran – Platform Administrator	Maintains servers, CI/CD and access	Manual user set-up, inconsistent pipelines	Automated, secure, standard pipelines

1.7 Scope, Assumptions and Constraints

Category	Details
In scope	Teams and tasks, requirements and documents, Git integration and code review, automated testing, CI/CD, containerized environments, dashboards, risk tracking, role-based access, audit trail
Out of scope (first release)	Mobile application, payment features, video conferencing, plagiarism detection, integration with the college ERP
Assumptions	Teams use Git repositories; students have internet access; Docker is available on the servers; faculty act as Product Owner
Constraints	Academic-semester timeline (4 sprints); free/open-source tools preferred; limited server resources; student team skills vary

1.8 High-Level Solution Components

Component	Purpose	Suggested Technology
Web Front-end	Task board, dashboards, review screens	React / HTML5, CSS3
Application Server (API)	Business logic, authentication, role checks	Node.js / Java Spring Boot, REST APIs
Database	Users, teams, tasks, requirements, audit logs	PostgreSQL / MySQL
Version Control Integration	Repositories, branches, pull requests, webhooks	Git, GitHub/GitLab API
CI/CD and Testing Service	Automated builds, tests and deployment	Jenkins / GitHub Actions, JUnit / PyTest
Container Service	Reproducible development environments	Docker, Docker Compose
Notification and Dashboard Service	Alerts, burndown and progress charts	WebSockets, Chart.js

2. Epics and User Stories

2.1 Epics

Epic ID	Epic	Description
E1	Team, Task and Milestone Management	Create teams, roles, tasks, issues and milestones
E2	Requirements and Documentation	Collaborative requirements and project documents with traceability
E3	Version Control and Code Review	Git integration, branching, pull requests, reviews and conflict prevention

Epic ID	Epic	Description
E4	Automated Testing and CI/CD	Automatic builds, tests and deployment pipelines
E5	Containerized Development Environments	Ready-to-use, consistent container-based workspaces
E6	Progress, Risk and Dashboards	Progress tracking, burndown, risk register and alerts
E7	Access Control, History and Audit	Role-based access, project history and audit trails

2.2 User Stories

Format: **As a** <user>, **I want** <feature>, **so that** <benefit>.

ID	Epic	User Story
US-01	E1	As a project manager, I want to create teams and assign roles and responsibilities, so that every member knows their duties.
US-02	E1	As a team lead, I want to create tasks with an assignee, priority and due date, so that work is not duplicated and deadlines are clear.
US-03	E1	As a project manager, I want to define milestones and link tasks to them, so that I can track deadlines across teams.
US-04	E2	As a documentation lead, I want to edit requirements and documents collaboratively with version history, so that project documentation stays consistent.
US-05	E2	As a business analyst, I want to link requirements to tasks and test cases, so that every requirement is traceable.
US-06	E3	As a developer, I want to connect my team repository and create feature branches linked to tasks, so that my code contributions are organised and traceable.
US-07	E3	As a reviewer, I want to review pull requests with comments and approve or reject them, so that only quality code is merged.
US-08	E3	As a developer, I want to be warned about possible merge conflicts before merging, so that conflicts are resolved early.
US-09	E4	As a tester, I want automated tests to run on every push, so that defects are detected early.
US-10	E4	As a team lead, I want a CI/CD pipeline that builds, tests and deploys the project with status reports, so that releases are reliable.
US-11	E5	As a developer, I want a one-click containerized development environment, so that setup is quick and identical for all team members.
US-12	E6	As a faculty guide, I want a dashboard showing team progress, burndown and milestone status, so that I can monitor all teams easily.
US-13	E6	As a project manager, I want to log risks and receive alerts for overdue tasks, so that problems are handled early.
US-14	E7	As an administrator, I want role-based access control, so that only authorised users can perform specific actions.
US-15	E7	As a faculty evaluator, I want an audit trail and project history, so that I have evidence for assessment.

2.3 INVEST Check for the Sprint 1 Stories

Each Sprint 1 story was checked against the INVEST principles (Independent, Negotiable, Valuable, Estimable, Small, Testable).

Story	I	N	V	E	S	T	Remark
US-14	Yes	Yes	Yes	Yes	Yes	Yes	Foundation story; no dependency
US-01	Yes	Yes	Yes	Yes	Yes	Yes	Small CRUD story
US-02	Partly	Yes	Yes	Yes	Yes	Yes	Needs teams (US-01) first; ordered accordingly
US-06	Partly	Yes	Yes	Yes	Partly	Yes	Largest story; kept at 8 points, can be split if needed
US-07	Partly	Yes	Yes	Yes	Yes	Yes	Depends on Git integration
US-09	Partly	Yes	Yes	Yes	Yes	Yes	Depends on repository connection

2.4 Non-Functional Requirements

Category	Requirement	Measure
Performance	Pages and APIs respond quickly under normal load	95% of requests within 2 seconds for 200 concurrent users
Security	Passwords hashed, sessions protected, role checks on every request	No unauthorised access in testing; HTTPS only
Availability	Platform available during academic working hours	At least 99% uptime during the semester
Usability	Simple interface for first-time users	New user completes first task creation in under 3 minutes
Scalability	Support many teams and repositories	At least 50 teams and 500 users
Maintainability	Clean modular code with documentation	Code coverage of at least 70%; coding standard followed
Auditability	All important actions are logged	Every create/update/delete action stored with user and time

3. Acceptance Criteria (Given–When–Then)

Acceptance criteria are written for the six user stories selected for Sprint 1 (Section 5).

US-14 – Role-Based Access Control

- **Given** a registered user with the role "Student", **when** they try to open the admin settings page, **then** access is denied with a 403 message in under 2 seconds.
- **Given** an administrator is logged in, **when** they assign the role "Reviewer" to a user, **then** the new permissions take effect on the next request and the change is recorded.

- **Given** a user enters an incorrect password 5 times, **when** they try a sixth time, **then** the account is locked for 15 minutes.

US-01 – Create Teams and Assign Roles

- **Given** a project manager is logged in, **when** they create a team with a name and at least 1 member, **then** the team appears in the team list immediately.
- **Given** a team exists, **when** the manager assigns a role (Lead, Developer, Tester, Documenter) to a member, **then** the role is shown on the member's profile.
- **Given** a member already belongs to a team, **when** the manager tries to add them again, **then** the system shows a "duplicate member" error.

US-02 – Create and Assign Tasks

- **Given** a team lead is on the task board, **when** they submit a task with title, assignee, priority and due date, **then** the task is created in the "To Do" column.
- **Given** a task is created without a title, **when** the lead submits it, **then** the system rejects it and highlights the missing field.
- **Given** a task is assigned to a member, **when** the assignment is saved, **then** that member receives a notification within 1 minute.

US-06 – Git Repository Integration and Branching

- **Given** a team lead provides a valid repository URL, **when** they click "Connect", **then** the repository is linked and the commit history loads within 10 seconds.
- **Given** a task exists, **when** a developer clicks "Create Branch", **then** a branch named feature/<task-id>-<title> is created and shown on the task.
- **Given** an invalid repository URL, **when** the user clicks "Connect", **then** a clear error message is displayed and nothing is linked.

US-07 – Pull Request Code Review

- **Given** a developer opens a pull request, **when** it is created, **then** at least one reviewer is auto-assigned and notified.
- **Given** a reviewer opens the pull request, **when** they add an inline comment and click "Approve", **then** the review status changes to "Approved" and is time-stamped.
- **Given** a pull request has no approval, **when** a user tries to merge it, **then** the merge button remains disabled.

US-09 – Automated Tests on Every Push

- **Given** a repository is connected, **when** a developer pushes a commit, **then** the test pipeline starts automatically within 30 seconds.
- **Given** a pipeline run finishes, **when** all tests pass, **then** the commit shows a green "Passed" status; **if** any test fails, **then** it shows "Failed" with the log link.
- **Given** a test run fails, **when** the pull request page is opened, **then** merging is blocked until the tests pass.

3.2 Acceptance Criteria for Remaining Backlog Stories

Story	Acceptance Criteria
US-03	**Given** a manager creates a milestone with a due date, **when** tasks are linked to it, **then** the milestone shows the percentage of completed tasks. **Given** the due date passes with open tasks, **then** the milestone is marked "At Risk".
US-04	**Given** two members edit a document, **when** both save, **then** both versions are stored and no content is lost. **Given** a document has versions, **when** the user selects an older version, **then** it can be restored in one click.
US-05	**Given** a requirement exists, **when** a task or test case is linked to it, **then** the requirement page lists all linked items. **Given** a requirement has no test case, **then** it is flagged "Not covered".
US-08	**Given** two branches modify the same file lines, **when** a pull request is created, **then** a conflict warning is shown within 10 seconds naming the files.
US-10	**Given** a pull request is merged to main, **when** the pipeline runs, **then** it builds, tests and deploys to staging, and shows each stage status. **Given** any stage fails, **then** deployment stops and the team lead is notified.
US-11	**Given** a developer clicks "Start Environment", **when** the container is created, **then** it is ready with the project dependencies in under 10 minutes and the same on every machine.
US-12	**Given** a faculty guide opens the dashboard, **when** the page loads, **then** it shows per-team progress, burndown chart and milestone status within 3 seconds.
US-13	**Given** a risk is logged with probability and impact, **when** the risk score is high, **then** it is highlighted in red. **Given** a task passes its due date, **then** an alert is sent to the lead.
US-15	**Given** any create, update or delete action, **when** it occurs, **then** an entry with user, action and time-stamp is stored and cannot be edited by normal users.

3.3 Definition of Ready

- The story follows the "As a… I want… so that…" format and meets the INVEST checks.
- Acceptance criteria are written and agreed with the Product Owner.
- Dependencies are identified and the story is estimated by the team.
- The story is small enough to be finished within one sprint.

4. Prioritized Product Backlog

The backlog is ordered using **business value**, **customer (faculty/student) need** and **technical dependencies** (for example, access control and teams must exist before tasks, Git and CI/CD). MoSCoW is used for priority: **M** = Must have, **S** = Should have, **C** = Could have.

Rank	ID	Backlog Item	Business Value	Depends On	MoSCoW	Points	Planned
1	US-14	Role-based access control	High	None (foundation)	M	5	Sprint 1
2	US-01	Create teams and roles	High	US-14	M	3	Sprint 1
3	US-02	Create and assign tasks	High	US-01	M	3	Sprint 1

Rank	ID	Backlog Item	Business Value	Depends On	MoSCoW	Points	Planned
4	US-06	Git repository integration	High	US-01, US-02	M	8	Sprint 1
5	US-07	Pull request code review	High	US-06	M	5	Sprint 1
6	US-09	Automated tests on every push	High	US-06	M	5	Sprint 1
7	US-03	Milestones linked to tasks	Medium	US-02	M	5	Sprint 2
8	US-10	CI/CD pipeline	High	US-09	M	8	Sprint 2
9	US-08	Merge-conflict warning	Medium	US-06, US-07	S	5	Sprint 2
10	US-11	Containerized dev environment	Medium	US-06	S	8	Sprint 2
11	US-12	Progress dashboard	High	US-02, US-03	S	8	Sprint 3
12	US-04	Collaborative documentation	Medium	US-14	S	8	Sprint 3
13	US-05	Requirement traceability	Medium	US-04, US-02	S	5	Sprint 3
14	US-13	Risk register and alerts	Medium	US-03	C	5	Sprint 4
15	US-15	Audit trail and history	Medium	US-14	C	5	Sprint 4

Justification: US-14, US-01 and US-02 come first because every other feature (repositories, reviews, tests, dashboards) needs authenticated users, teams and tasks. Git integration, code review and automated testing follow because they directly solve the core problems of merge conflicts and poor code quality. Dashboards, documentation and audit features are delivered after the data they depend on exists.

4.2 Release Plan

Sprint	Theme	Stories	Points	Release Outcome
Sprint 1	Core foundation	US-14, US-01, US-02, US-06, US-07, US-09	29	Secure users, teams, tasks, Git and automated tests
Sprint 2	Delivery pipeline	US-03, US-10, US-08, US-11	26	Milestones, CI/CD, conflict warning, containers
Sprint 3	Visibility and documentation	US-12, US-04, US-05	21	Dashboards, documents, traceability
Sprint 4	Governance and release	US-13, US-15, hardening	10 + fixes	Risk tracking, audit trail, final testing and demo

5. Sprint Backlog (Sprint 1)

Sprint length: 2 weeks (10 working days) **Team size:** 5 members **Planned velocity:** 29 story points (6 stories). The team capacity is about 5 members × 10 days × 5 productive hours = 250 hours; planned task effort is about 180 hours, leaving a buffer for reviews, meetings and defects.

User Story	Task	Owner (Role)	Est. Hours
US-14 (5 pts)	T1 Design user/role database schema	Backend Dev	6
	T2 Implement login, registration and JWT authentication	Backend Dev	10
	T3 Implement role permissions (Admin, Manager, Lead, Student, Reviewer)	Backend Dev	8
	T4 Build login and role-based UI screens; test	Frontend Dev / QA	8
US-01 (3 pts)	T5 Create team and member APIs	Backend Dev	6
	T6 Build team management screen with role assignment	Frontend Dev	6
	T7 Write unit tests for team module	QA	3
US-02 (3 pts)	T8 Create task model and APIs (create, assign, update)	Backend Dev	6
	T9 Build Kanban task board UI	Frontend Dev	8
	T10 Add assignment notifications and tests	Backend Dev / QA	5
US-06 (8 pts)	T11 Study Git provider API and set up access tokens	DevOps	6
	T12 Implement repository connect and commit-history sync	Backend Dev	14
	T13 Implement branch creation linked to task ID	Backend Dev	8
	T14 Build repository and branch UI; integration tests	Frontend Dev / QA	10
US-07 (5 pts)	T15 Pull request create/list APIs with auto reviewer assignment	Backend Dev	10
	T16 Review UI: inline comments, approve/reject, merge lock	Frontend Dev	12
	T17 Test review workflow end to end	QA	5
US-09 (5 pts)	T18 Set up CI runner and webhook on push	DevOps	8
	T19 Create sample unit test suite and pipeline script	QA / DevOps	8
	T20 Show pass/fail status and log link; block merge on failure	Backend / Frontend	10
Team	T21 Sprint setup: repo, coding standards, Docker base, documentation	All	12

Total estimated effort: 180 hours (approximately).

5.2 Day-by-Day Sprint Plan

Day	Planned Activity	Output
Day 1	Sprint planning, task allocation, environment and repository set-up	Sprint backlog, base project
Day 2	Database schema; authentication design (US-14)	Schema, auth design
Day 3	Login, roles and permissions; team APIs (US-14, US-01)	Working login and teams API
Day 4	Team screen; task APIs (US-01, US-02)	Team UI, task API
Day 5	Kanban board; Git provider study (US-02, US-06)	Task board, Git access tokens
Day 6	Repository connection and commit sync; branch creation (US-06)	Linked repositories
Day 7	Pull request APIs and review screens (US-07)	Review workflow
Day 8	CI runner, webhook and test pipeline (US-09)	Pipeline on push
Day 9	Integration testing, defect fixing, documentation	Test report
Day 10	Sprint review demo and retrospective	Demo, retrospective notes

5.3 Sprint 1 Test Plan

Test Type	Scope	Responsible	Tool
Unit testing	Services and APIs of all six stories	Developers	JUnit / PyTest
Integration testing	Login → team → task → branch → pull request flow	QA	Postman
Security testing	Role checks, locked accounts, unauthorised access	QA / DevOps	Manual, OWASP checklist
User acceptance testing	All acceptance criteria demonstrated	Product Owner	Demo checklist

6. Story Point Estimation

Technique: Planning Poker using the modified Fibonacci scale (1, 2, 3, 5, 8, 13). The team estimated by comparing each story with a **reference story**, US-02 "Create and assign tasks", which is a standard create-read-update feature with a simple UI and was fixed at **3 points**. Estimates consider effort, complexity, uncertainty and testing. After the first round of voting, members with the highest and lowest estimates explained their reasons and the team re-voted until agreement was reached.

Story	Points	Justification
US-14	5	Authentication and permissions are well known but security-critical; touches every module and needs thorough testing.
US-01	3	Simple data model and CRUD screens; comparable in size to the reference story.
US-02	3	**Reference story:** standard CRUD with a board UI and a notification.
US-06	8	Depends on an external Git provider API, tokens, syncing history and branch automation; highest uncertainty in the sprint.

Story	Points	Justification
US-07	5	Moderate complexity (reviewers, comments, approval states, merge lock) but builds on the Git integration.
US-09	5	Needs CI runner, webhook and status reporting; known tools but pipeline set-up adds some risk.

Total Sprint 1 commitment = 5 + 3 + 3 + 8 + 5 + 5 = 29 story points, which is within the assumed team velocity of 28–30 points per 2-week sprint.

6.2 Planning Poker Rounds

Five team members (Developer 1, Developer 2, Tester, DevOps, Team Lead) voted privately with Fibonacci cards.

Story	Round 1 Votes	Discussion	Round 2 Votes	Agreed
US-14	5, 5, 8, 3, 5	Security testing effort explained	5, 5, 5, 5, 5	5
US-01	3, 2, 3, 3, 5	Role assignment screen clarified	3, 3, 3, 3, 3	3
US-02	3, 3, 3, 2, 3	Reference story; quick consensus	3, 3, 3, 3, 3	3
US-06	8, 13, 8, 5, 8	External API risk and token handling	8, 8, 8, 8, 8	8
US-07	5, 3, 5, 8, 5	Merge lock and comments discussed	5, 5, 5, 5, 5	5
US-09	5, 8, 5, 5, 3	Pipeline template reduces effort	5, 5, 5, 5, 5	5

6.3 Ideal Burndown for Sprint 1

Day	1	2	3	4	5	6	7	8	9	10
Points remaining	29	26	23	20	17	14	11	8	4	0

The burndown chart is updated every day during the Daily Scrum. If the actual line stays above the ideal line for two days, the team lead discusses scope or help with the Product Owner.

7. Sprint Goal and Expected Deliverables

7.1 Sprint Goal

"By the end of Sprint 1, a secure working version of the platform will allow authorised users to create teams, assign tasks, connect a Git repository, review pull requests and see automated test results on every code push."

7.2 Expected Deliverables (Increment)

- Secure login with role-based access control (Admin, Manager, Team Lead, Student, Reviewer).
- Team management module with member roles and responsibilities.
- Task board (To Do, In Progress, Done) with assignment and notifications.

- Git repository integration with commit history and task-linked branches.
- Pull request code review workflow with approve/reject and merge restrictions.
- Automated test pipeline triggered on every push, showing pass/fail status.
- Sprint documentation: API documentation, test report, updated backlog and burndown chart.

7.3 Definition of Done

- Code is peer-reviewed and merged into the main branch.
- All acceptance criteria are met and unit/integration tests pass in the pipeline.
- No open critical or high-severity defects.
- Documentation is updated and the feature is demonstrated in the Sprint Review.

7.4 Key Risks and Mitigation

Risk	Impact	Mitigation
Git provider API changes or rate limits	Delays in US-06	Study the API early (T11), cache responses and use a mock for testing
Team members inexperienced with CI/CD	US-09 may slip	Pair a DevOps member with QA; use a ready-made pipeline template
Scope creep from extra requests	Sprint goal missed	Product Owner approves changes; new items go to the backlog

7.5 Scrum Events for the Sprint

- **Sprint Planning:** Day 1 (selection of stories, tasks and estimates).
- **Daily Scrum:** 15 minutes every day.
- **Sprint Review:** Day 10 (demo to the faculty guide).
- **Sprint Retrospective:** Day 10 (what went well, what to improve).

7.6 Sprint Review and Retrospective Plan

Activity	Focus	Output
Sprint Review	Demonstrate each completed story against its acceptance criteria to the faculty guide	Accepted / rejected stories, new backlog items
Retrospective – Went well	Teamwork, pairing, early testing	Practices to continue
Retrospective – To improve	Estimation accuracy, review turn-around time	Improvement actions for Sprint 2
Backlog Refinement	Re-estimate and re-prioritise Sprint 2 stories	Updated product backlog

8. Conclusion

This sprint planning document applies Agile and Scrum practices to Problem 35, the Distributed Online Collaboration Platform for Large Student Projects. The vision and stakeholders were identified, requirements were broken into epics and INVEST-checked user stories with measurable Given–When–

Then acceptance criteria, and the product backlog was prioritised using business value, customer need and dependencies. Six stories worth 29 story points were planned into Sprint 1 with a task breakdown, day-by-day plan and test plan, estimated using Planning Poker. Delivering this sprint will give student teams a secure, working base for task management, Git-based collaboration, code review and automated testing, which directly reduces duplicated work and merge conflicts and improves project visibility.

Submitted by: S. Roshni (192525185) **Signature:** ***S.Roshni***

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Project Vision & Stakeholder Identification (30%)	Project vision is clear, comprehensive, and aligned with the problem statement. All relevant stakeholders and their roles are correctly identified.	Vision is clear with minor omissions. Most stakeholders are correctly identified.	Vision is partially defined. Some stakeholders are missing or incorrectly identified.	Vision is unclear or incomplete. Stakeholders are poorly identified or missing.
Epics & User Stories (25%)	Epics are well-defined and user stories are complete, follow Agile format, and fully capture functional requirements.	Epics and user stories are mostly complete with minor formatting or requirement issues.	User stories are partially complete with limited Agile structure.	User stories are incomplete, unclear, or do not follow Agile practices.
Acceptance Criteria (20%)	Acceptance criteria are specific, measurable, testable, and aligned with all user stories.	Acceptance criteria are mostly clear and testable with minor gaps.	Acceptance criteria are partially defined and lack clarity or completeness.	Acceptance criteria are missing or not aligned with user stories.
Product Backlog Prioritization & Sprint Planning (15%)	Product backlog is logically prioritized, sprint backlog is realistic, and task breakdown is complete.	Backlog prioritization and sprint planning are mostly appropriate with minor issues.	Prioritization or sprint planning is partially complete with limited justification.	Backlog prioritization and sprint planning are poorly organized or incomplete.
Story Point Estimation, Sprint Goal & Presentation (10%)	Story point estimation is well justified, sprint goal is clear, deliverables are realistic, and presentation is professional and well organized.	Estimation and sprint goal are mostly appropriate with minor inconsistencies. Presentation is clear.	Estimation or sprint goal lacks justification. Presentation is adequate but incomplete.	Estimation is unrealistic or missing. Sprint goal, deliverables, and presentation are unclear or incomplete.

Criteria	Mark
Project Vision & Stakeholder Identification **(30%)**	/5
Epics & User Stories **(25%)**	/5
Acceptance Criteria **(20%)**	/5
Product Backlog Prioritization & Sprint Planning **(15%)**	/5
Story Point Estimation, Sprint Goal & Presentation **(10%)**	/5
TOTAL	/25